

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a340c346284071bd8.jsonl`  
- SHA-256: `c4736641769a402602abf9df7b5cc4df20d64c88472b2ba15de2dffe2f0871a7`  
- Source modified: `2026-07-06T19:19:41+00:00`  
- Imported at: `2026-07-08T16:00:11+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:18:19.569Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (design): "Extra fields are ignored" means DROPPED, not preserved ? the entire config-storage design silently loses topics/allowlist/deep_inspect severity high | section Decision ? Schema ? minimal changes; and Open questions (backwards-compat claim)
summary: The ADR's load-bearing claim that new keys are 'backwards-compatible with the existing FilterConfig Pydantic model (extra fields are ignored)' is inverted: Pydantic's default extra='ignore' DISCARDS unknown keys, so topics/allowlist/security_checks/deep_inspect never survive a save.
why_it_matters: The whole design stores Tier 2/3/4 config as new keys inside policies.config_json and relies on FilterConfig tolerating them. But the persistence path is save_filter_config() -> Policy(config_json=cfg.model_dump_json()) where cfg is a FilterConfig with model_config={} (i.e. extra='ignore'). Verified empirically: FilterConfig.model_validate({'min_duration_s':5,'topics':{'...},'deep_inspect':True}).model_dump_json() emits ONLY the known FilterConfig fields ? topics/deep_inspect are gone. So any policy config written through the shim silently drops every Track-F key. F2/F3/F4/F5 all read config that the save path deletes. This breaks the phases, not just an imprecise sentence.
evidence: models.py FilterConfig has no model_config extra setting (defaults to 'ignore'); `FilterConfig.model\_config` prints `{}`. db.py:1711 save_filter_config does `config\_json=cfg.model\_dump\_json()`. Round-trip test: persisted blob = only the 9 FilterConfig fields; 'topics survived round-trip: False'.
rec: The ADR must specify HOW Track-F keys are persisted. Either (a) extend the FilterConfig model with typed optional topics/allowlist/security_checks/deep_inspect fields (then it's a schema change to models.py, contradicting 'code concept only, extra fields ignored'), or (b) stop routing config through FilterConfig and store/parse the raw config_json dict directly, or (c) set model_config = ConfigDict(extra='allow'). State which, because 'ignored' as written guarantees data loss.

2. assistant (2026-07-06T19:18:21.882Z)

I'll verify this finding by examining the ADR and the actual code behavior.

3. user (2026-07-06T19:18:23.432Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

```
| Concept | Current state |
|---|---|
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig`
blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were
deferred to v2 in the original plan. |
```

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                               Policy Pipeline                       ?
?                               ?                                     ?
? Tier 1 ? Metadata gates (always runs, zero I/O)                   ?
?   duration, resolution, size, codec, mime, GIF/round              ?
?   ? extends today's FilterEngine                                  ?
?                               ?                                     ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)                ?
?   source allowlist, keyword match, channel topic tags            ?
?   ? user-expressed interests filter the stream                   ?
?                               ?                                     ?
? Tier 3 ? Content safety (opt-in, metadata + light I/O)            ?
?   URL/domain reputation, file metadata sanity,                   ?
?   caption text scanning                                           ?
?   ? cheap checks; no download                                     ?
?                               ?                                     ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)                      ?
?   ffprobe, perceptual hash, downloaded content scan              ?
?   ? gated behind explicit user opt-in + budget                   ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
```

```

 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
...

```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```

```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
```
```

The caller (telethon\_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

## Schema ? minimal changes

**\*\*No schema migration\*\*** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

## Implementation phasing (future issues)

| Phase   | Scope                   | Dependencies                                                                  |
|---------|-------------------------|-------------------------------------------------------------------------------|
| ***F1** | Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler |
| ***F2** | Topic preferences       | `topics` config parsing, source title/caption keyword matching                |
| ***F3** | Source allowlist        | `allowlist` config, skip non-allowlisted sources                              |
| ***F4** | URL/domain reputation   | Domain blacklist, caption link extraction                                     |
| ***F5** | Deep inspection         | ffprobe/perceptual hash, download budget enforcement                          |
| ***F6** | Policy decisions table  | Audit trail storage, explainability UI                                        |
| ***F7** | Explainability endpoint | `/why <content_item_id>` command showing the explanation trace                |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the **\*\*design contract\*\*** the phases code against.

## Consequences

### **\*\*Positive\*\***

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

## **\*\*Negative\*\***

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

## **Alternatives considered**

- **\*\*Do nothing; keep flat FilterConfig.\*\*** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- **\*\*Inline each stage in the handler\*\*** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- **\*\*Use a rules engine / DSL.\*\*** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- **\*\*Bundle all phases into a single PR.\*\*** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

## **Open questions for the decision owner**

1. **\*\*`StageResult.metadata` shape\*\*** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. **\*\*Policy decisions table\*\*** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. **\*\*Topic matching\*\*** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. **\*\*Deep inspection budget\*\*** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

## **References**

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

### **4. assistant (2026-07-06T19:18:25.279Z)**

Now let me verify the core claim empirically by checking the actual `FilterConfig` model behavior.

### **5. user (2026-07-06T19:18:27.505Z)**

main

c4c4d18 fix(scanning): don't clobber user-controlled watched on source upsert (#41)

## 6. user (2026-07-06T19:18:28.358Z)

```
"""Pydantic data models for users, channels, videos, and filter config.
```

```
These models are the in-memory representation used by the repository layer
(`db.py`) and the filter engine (`filters/`). They are deliberately
decoupled from Telethon's own types so the filter engine stays pure and
unit-testable without a live Telegram connection.
```

```
"""
```

```
from __future__ import annotations
```

```
from datetime import UTC, datetime
from enum import Enum
```

```
from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
```

```
def _utcnow() -> datetime:
 """Timezone-aware UTC 'now', used as a default factory."""
 return datetime.now(UTC)
```

```
class DeliveryMode(str, Enum):
 """How a matched video is delivered to the user.

 - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
 - ``LINK``: send a text message with a time link + metadata summary.
 - ``BOTH``: forward the video AND send the link message.
 """
```

```
 FORWARD = "forward"
 LINK = "link"
 BOTH = "both"
```

```
class Account(BaseModel):
 """A product identity ? the owner of config, channels, routes, and content.
```

```
Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
is the bridge used by the Bot API front-door (`from.id`) and by the
legacy userbot path; it is nullable so future providers that don't have a
Telegram id (e.g. WhatsApp) can still own an ``Account``.
"""
```

```
 model_config = ConfigDict(frozen=True)
```

```
 id: int = Field(description="Stable surrogate PK; the owner key for all downstream tables")
 telegram_id: int | None = Field(
 default=None,
 description="Telegram user id (bot from.id OR userbot account id). UNIQUE when set.",
)
 provider: str = Field(default="telegram", description="Provider namespace; future:
'whatsapp'")
 display_name: str | None = None
 created_at: datetime = Field(default_factory=_utcnow)
```

```
class TelegramSession(BaseModel):
 """An optional MTPProto (Telethon) session linked to an :class:`Account`.
```

```
A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
that is what makes them structurally invisible to ``manager.run``
(ADR 0010). ``session_path`` is data-driven so the file-location
convention is explicit rather than derived from the PK at runtime.
```

```

"""

model_config = ConfigDict(frozen=True)

account_id: int = Field(description="FK ? accounts.id; PK (one session per account in v1)")
phone: str | None = Field(default=None, description="E.164 phone; used only for bootstrap")
session_path: str = Field(description="File-backed Telethon session path
(user_<id>.session)")
enabled: bool = Field(
 default=True,
 description="If False, the manager skips this session (not the account)",
)
created_at: datetime = Field(default_factory=_utcnow)
last_login_at: datetime | None = None

class User(BaseModel):
 """Deprecated: legacy v1 model conflating identity + MTPROTO session.

 Retained as a backwards-compatible view over the new ``accounts`` +
 ``telegram_sessions`` tables (ADR 0010). New code should use
 :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
 maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
 and tests keep working without a flag day.
 """

 model_config = ConfigDict(frozen=True)

 id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy rows)")
 phone: str | None = Field(default=None, description="E.164 phone; used only for bootstrap")
 enabled: bool = Field(default=True, description="If False, the manager skips this session")
 created_at: datetime = Field(default_factory=_utcnow)

class Channel(BaseModel):
 """A channel/group a user is a member of. Populated lazily as messages arrive.

 Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
 code should use :class:`Source` directly; the ``db`` layer maps
 ``Channel`` ? ``Source`` so existing callers and tests keep working.
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 user_id: int
 channel_id: int = Field(description="Telegram channel/chat id")
 title: str | None = None
 seen_first_at: datetime = Field(default_factory=_utcnow)

```

### --- provider-neutral domain models (ADR 0011, Track B) -----

#

**These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig`` shapes so the schema no longer leaks Telegram-specific ids into every future provider's code path. ``provider`` columns + provider-prefixed ``dedup\_key`` make cross-provider collisions structurally impossible. See ADR 0011 for the full rationale and the two decisions (Option 1A: tiered ``dedup\_key``; Option 2A: ``policies`` generalizes ``filter\_config``).**

## 7. assistant (2026-07-06T19:18:31.123Z)

Now let me find the exact `FilterConfig` definition and its `model\_config`.

## 8. user (2026-07-06T19:18:32.983Z)

```
299:class FilterConfig(BaseModel):
```

## 9. user (2026-07-06T19:18:36.128Z)

```
class FilterConfig(BaseModel):
 """Per-user filter configuration.

 v1 rules: duration bounds, minimum height (resolution), and max file size.
 Stored in the DB as a JSON blob (`filter_config.config_json`) so adding a
 new field later is backwards-compatible.
 """

 # Duration bounds in seconds. None means unbounded on that side.
 min_duration_s: int | None = Field(default=5, ge=0)
 max_duration_s: int | None = Field(default=7200, ge=0)

 # Resolution: a video passes if its height >= target_height.
 # None disables the resolution rule.
 target_height: int | None = Field(default=1080, ge=1)

 # Max file size in mebibytes. None disables the size rule.
 max_size_mb: int | None = Field(default=2048, ge=1)

 # Content-type exclusions. GIFs and round videos are excluded by default
 # (their duration/dimensions are usually incidental to real video filters);
 # silent videos are off by default since some legit clips have no audio.
 # Existing users' stored configs are unaffected ? the DB JSON blob overrides
 # these defaults, so this only changes the behaviour for fresh installs.
 exclude_gif: bool = Field(
 default=True,
 description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
)
 exclude_round: bool = Field(
 default=True,
 description="If True, exclude circular video messages (round_message=True)",
)
 exclude_nosound: bool = Field(
 default=False,
 description="If True, exclude videos without an audio track (nosound=True)",
)

 # How matched videos are delivered: forward the video, send a link, or both.
 # Stored in the same JSON blob, so adding this field is backwards-compatible
 # (existing rows without it default to FORWARD).
 delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)

 # Where matched videos / link messages are sent. Accepts:
 # - "me" (default): Saved Messages ? exact v1.1 behaviour.
 # - "@username": a public channel the account is a member of.
 # - "-100..." or a bare int-as-string: a numeric Telegram chat id
 # (private channels without a username).
 # The string is passed verbatim to Telethon's forward_messages / send_message.
 # See ADR 0005 for rationale and access requirements.
 delivery_target: str = Field(
 default="me",
 min_length=1,
 description="Telethon entity to deliver to: 'me', '@username', or a chat id",
)
```

```

@field_validator("delivery_target", mode="before")
@classmethod
def _coerce_delivery_target(cls, v: object) -> str:
 """Accept int chat ids (e.g. -1001234567890) and coerce to str.

 Telethon accepts both str and int as an entity, but storing as str keeps
 the JSON blob uniform and lets '@username' / 'me' / numeric ids share
 one field type.
 """
 if v is None:
 return "me"
 if isinstance(v, int):
 return str(v)
 return str(v)

@model_validator(mode="after")
def _check_duration_bounds(self) -> FilterConfig:
 """Ensure min_duration_s <= max_duration_s when both are set."""
 if (
 self.min_duration_s is not None
 and self.max_duration_s is not None
 and self.min_duration_s > self.max_duration_s
):
 raise ValueError(
 f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s "
 f"({self.max_duration_s})"
)
 return self

class VideoRecord(BaseModel):
 """A seen video message + its metadata. Drives dedup and future /search.

 Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
 New code should use :class:`ContentItem`; the ``db`` layer maps
 ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 user_id: int
 channel_id: int = Field(description="Origin Telegram channel id")
 message_id: int = Field(description="Telegram message id within the channel")
 document_id: int | None = Field(
 default=None,
 description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
)

 # Metadata extracted from the Telethon message (may be None if absent).
 duration_s: int | None = None
 width: int | None = None
 height: int | None = None
 size_bytes: int | None = None
 mime_type: str | None = None

 matched: bool = False
 forwarded: bool = False
 seen_at: datetime = Field(default_factory=_utcnow)

 @property
 def dedup_key(self) -> tuple[int, int, int]:
 """The natural key used for deduplication in the ``videos`` table."""
 return (self.user_id, self.channel_id, self.message_id)

```

```

@property
def content_fingerprint(self) -> tuple[int, int, int, int] | None:
 """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height, width).

```

## 10. assistant (2026-07-06T19:18:38.529Z)

Confirmed: `FilterConfig` has NO `model\_config` set, so it defaults to `extra='ignore'`. Now let me verify empirically the round-trip behavior and check the save path.

## 11. user (2026-07-06T19:18:40.657Z)

```

83: config_json TEXT NOT NULL
132: config_json TEXT NOT NULL,
476: config_json TEXT NOT NULL
479: "user_id, config_json",
622: INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
624: SELECT user_id, 'default', config_json, 1,
695: CASE WHEN json_valid(config_json)
696: THEN json_extract(config_json, '$.delivery_target') END,
701: CASE WHEN json_valid(config_json)
702: THEN json_extract(config_json, '$.delivery_target') END,
726: CASE WHEN json_valid(fcd.config_json)
727: THEN json_extract(fcd.config_json, '$.delivery_target') END,
885: config_json=row["config_json"],
1253: """INSERT INTO policies (account_id, name, config_json, is_default,
1257: config_json=excluded.config_json,
1263: policy.config_json,
1698:async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
1708: return FilterConfig.model_validate_json(policy.config_json)
1711:async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:
1720: config_json=cfg.model_dump_json(),

```

## 12. user (2026-07-06T19:18:43.918Z)

```

async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
 """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.

```

```

ADR 0011 (Option 2A): ``filter_config`` is renamed to
``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
"""

```

```

policy = await get_default_policy(conn, user_id)
if policy is None:
 return FilterConfig()
return FilterConfig.model_validate_json(policy.config_json)

```

```

async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig) ->
None:

```

```

 """Deprecated shim: upsert the account's default :class:`Policy`."""
 existing = await get_default_policy(conn, user_id)
 await upsert_policy(
 conn,
 Policy(
 id=existing.id if existing is not None else None,
 account_id=user_id,
 name="default",
 config_json=cfg.model_dump_json(),
 is_default=True,
 created_at=(existing.created_at if existing is not None else _now()),
 updated_at=_now(),
),
),
)

```

## --- bulk helpers (used by the manager on startup) -----

### 13. assistant (2026-07-06T19:18:46.592Z)

Now the critical empirical test. Let me verify the round-trip actually drops the extra keys.

### 14. user (2026-07-06T19:18:50.731Z)

```
model_config: {}
dumped: {"min_duration_s":5,"max_duration_s":7200,"target_height":1080,"max_size_mb":2048,"excl
de_gif":true,"exclude_round":true,"exclude_nosound":false,"delivery_mode":"forward","delivery_ta
rget":"me"}
topics survived: False
deep_inspect survived: False
allowlist survived: False
security_checks survived: False
```

### 15. assistant (2026-07-06T19:18:55.414Z)

Empirically confirmed. `model\_config` is `{}` (defaults to `extra='ignore'`), and all four Track-F keys (topics, allowlist, security\_checks, deep\_inspect) are silently dropped on `model\_dump\_json()`.

Now let me adversarially test the strongest refutation: maybe the raw `config\_json` stored in the DB retains extra keys, and only the \*shim\* strips them. If Track-F code reads `policy.config\_json` raw (not via FilterConfig), the keys could survive. Let me check what the ADR actually claims and whether the save path necessarily goes through FilterConfig.

### 16. user (2026-07-06T19:18:57.717Z)

```
"""Insert or update a policy row by `(account_id, name)`.
```

```
Exactly one policy per account should carry `is_default=True`; callers
are responsible for flipping the previous default before flipping a new
one (enforced by a partial unique index landed by future work, not in the
schema ? SQLite `CHECK` can't express "at most one per group").
"""
```

```
await conn.execute(
 """INSERT INTO policies (account_id, name, config_json, is_default,
 created_at, updated_at)
 VALUES (?, ?, ?, ?, ?, ?)
 ON CONFLICT(account_id, name) DO UPDATE SET
 config_json=excluded.config_json,
 is_default=excluded.is_default,
 updated_at=excluded.updated_at""",
 (
 policy.account_id,
 policy.name,
 policy.config_json,
 int(policy.is_default),
 policy.created_at.isoformat(),
 policy.updated_at.isoformat(),
),
)
await conn.commit()
conn.row_factory = aiosqlite.Row
```

### 17. assistant (2026-07-06T19:19:02.507Z)

`upsert\_policy` writes `policy.config\_json` verbatim as a string ? it does NOT re-validate through FilterConfig. So the \*raw\* `Policy.config\_json` path preserves extra keys. The question is whether the ADR's claim is about that raw path or the FilterConfig path.

Let me re-read the exact ADR wording. The ADR says the keys are "backwards-compatible with the

existing FilterConfig Pydantic model (extra fields are ignored)." This is the load-bearing sentence. Let me assess both the write and read paths carefully.

#### 18. user (2026-07-06T19:19:07.789Z)

RAW path topics survive: True  
via load\_filter\_config, topics accessible: False  
via load\_filter\_config dumped: False

#### 19. assistant (2026-07-06T19:19:22.740Z)

This clarifies the picture. Let me now weigh the finding against the ADR's actual claims and phrasing to reach a verdict.

The key facts:

1. The ADR's literal justification ? "backwards-compatible with the existing FilterConfig Pydantic model (extra fields are ignored)" ? is **\*\*inverted as stated\*\***. "Ignored" in Pydantic means dropped, not preserved. The empirical test confirms FilterConfig drops all four keys.
2. **\*\*However\*\***, the design does NOT strictly require the FilterConfig path. The raw ``Policy.config_json`` string round-trips fine through ``upsert_policy`` (verbatim write) and could be read as a raw dict. The ADR even says "The pipeline is a code concept" and lists reading ``topics`/`allowlist`` config in F2/F3/F4.
3. The **\*\*real gap\*\***: the ADR never specifies HOW Track-F code writes/reads these keys. If any write goes through the existing ``save_filter_config`` shim (the only documented persistence path in the current codebase, and the one the ADR points to via "the existing FilterConfig Pydantic model"), the keys are destroyed. The ADR explicitly leans on FilterConfig tolerance as its backwards-compat argument, and that argument is factually wrong.

Let me verify one more refutation angle: does the ADR anywhere say config is stored/read as a raw dict rather than FilterConfig? Re-reading ? no. It only names FilterConfig and asserts "extra fields are ignored" as the compat mechanism. There's no F-phase issue for "extend FilterConfig" or "parse raw config\_json."

The finding is essentially correct on the factual inaccuracy (the "ignored = preserved" inversion is a genuine false claim), but slightly overstated on "guarantees data loss" ? data loss only occurs on the FilterConfig write path, not the raw config\_json path, and the design doesn't **\*mandate\*** the FilterConfig path (it's just the only one that exists and the one the ADR cites). This makes it a real, load-bearing inaccuracy that must be resolved before F2-F5, but the "entire config-storage design silently loses" framing is somewhat too absolute.

#### 20. user (2026-07-06T19:19:41.631Z)

Structured output provided successfully